

Lecture (05)

Introduction to Classes and Objects

Topics

- 7.1 Abstract Data Types
- 7.2 Object-Oriented Programming
- 7.3 Introduction to Classes
- 7.4 Creating and Using Objects
- 7.5 Defining Member Functions
- 7.6 Constructors
- 7.7 Destructors
- 7.8 Private Member Functions
- 7.9 Passing Objects to Functions
- 7.10 Object Composition
- 7.11 Separating Class Specification, Implementation,
and Client Code

7.1 Abstract Data Types

- Programmer-created data types that specify
 - legal values that can be stored
 - operations that can be done on the values
- The user of an abstract data type (ADT) does not need to know any implementation details (*e.g.*, how the data is stored or how the operations on it are carried out)

Abstraction in Software Development

- Abstraction allows a programmer to design a solution to a problem and to use data items without concern for how the data items are implemented
- This has already been encountered in the book:
 - To use the **pow** function, you need to know what inputs it expects and what kind of results it produces
 - You do not need to know how it works

Abstraction and Data Types

- **Abstraction**: a definition that captures general characteristics without details
ex: An abstract triangle is a 3-sided polygon. A specific triangle may be scalene, isosceles, or equilateral
- **Data Type**: defines the kind of values that can be stored and the operations that can be performed on it

7.2 Object-Oriented Programming

- **Procedural programming** uses variables to store data, and focuses on the processes/ functions that occur in a program. Data and functions are separate and distinct.
- **Object-oriented programming** is based on objects that encapsulate the data and the functions that operate on it.

Object-Oriented Programming

Terminology

- **object**: software entity that combines data and functions that act on the data in a single unit
- **attributes**: the data items of an object, stored in **member variables**
- **member functions (methods)**: procedures/ functions that act on the attributes of the class

More Object-Oriented Programming Terminology

- **data hiding**: restricting access to certain members of an object. The intent is to allow only member functions to directly access and modify the object's data
- **encapsulation**: the bundling of an object's data and procedures into a single entity

Object Example

Square

Member variables (attributes)

```
int side;
```

Member functions

```
void setSide(int s)
{   side = s;   }
```

```
int getSide()
{   return side; }
```

Square object's data item: **side**

Square object's functions: **setSide** - set the size of the side of the square, **getSide** - return the size of the side of the square

Why Hide Data?

- Protection – Member functions provide a layer of protection against inadvertent or deliberate data corruption
- Need-to-know – A programmer can use the data via the provided member functions. As long as the member functions return correct information, the programmer needn't worry about implementation details.

7.3 Introduction to Classes

- **Class**: a programmer-defined data type used to define objects
- It is a pattern for creating objects

ex:

```
string fName, lName;
```

creates two objects of the **string** class

Introduction to Classes

- Class declaration format:

```
class className  
{  
    declaration;  
    declaration;  
};
```



Notice the
required ;

Access Specifiers

- Used to control access to members of the class.
- Each member is declared to be either
 - public**: can be accessed by functions outside of the class
 - or
 - private**: can only be called by or accessed by functions that are members of the class

Class Example

```
class Square
{
    private:
    int side;
    public:
    void setSide(int s)
    { side = s; }
    int getSide()
    { return side; }
};
```

Access
specifiers



More on Access Specifiers

- Can be listed in any order in a class
- Can appear multiple times in a class
- If not specified, the default is **private**

7.4 Creating and Using Objects

- An **object** is an instance of a class
- It is defined just like other variables

```
Square sq1, sq2;
```

- It can access members using dot operator

```
sq1.setSide(5);
```

```
cout << sq1.getSide();
```

Types of Member Functions

- **Acessor, get, getter function:** uses but does not modify a member variable
ex: `getSide`
- **Mutator, set, setter function:** modifies a member variable
ex: `setSide`

7.5 Defining Member Functions

- Member functions are part of a class declaration
 - Can place entire function definition inside the class declaration
- or
- Can place just the prototype inside the class declaration and write the function definition after the class

Defining Member Functions Inside the Class Declaration

- Member functions defined inside the class declaration are called **inline functions**
- Only very short functions, like the one below, should be inline functions

```
int getSide()  
{ return side; }
```

Inline Member Function Example

```
class Square
{
    private:
        int side;
    public:
        void setSide(int s)
        { side = s; }
        int getSide()
        { return side; }
};
```



A red oval containing the text "inline functions" has two red arrows pointing to the `setSide` and `getSide` member functions in the `Square` class definition.

Defining Member Functions After the Class Declaration

- Put a function prototype in the class declaration
- In the function definition, precede the function name with the class name and **scope resolution operator** (`::`)

```
int Square::getSide()  
{  
    return side;  
}
```

Conventions and a Suggestion

Conventions:

- Member variables are usually **private**
- Accessor and mutator functions are usually **public**
- Use 'get' in the name of accessor functions, 'set' in the name of mutator functions

Suggestion: calculate values to be returned in accessor functions when possible, to minimize the potential for stale data

Tradeoffs of Inline vs. Regular Member Functions

- When a regular function is called, control passes to the called function
 - the compiler stores return address of call, allocates memory for local variables, etc.
- Code for an inline function is copied into the program in place of the call when the program is compiled
 - This makes a larger executable program, but
 - There is less function call overhead, and possibly faster execution

7.6 Constructors

- A **constructor** is a member function that is often used to initialize data members of a class
- Is called automatically when an object of the class is created
- It must be a **public** member function
- It must be named the same as the class
- It must have no return type

Constructor – 2 Examples

Inline:

```
class Square
{
    . . .
public:
    Square(int s)
    { side = s; }
    . . .
};
```

Declaration outside the class:

```
Square(int);    //prototype
                //in class

Square::Square(int s)
                {
                side = s;
                }
```

Overloading Constructors

- A class can have more than 1 constructor
- Overloaded constructors in a class must have different parameter lists

```
Square () ;
```

```
Square (int) ;
```

The Default Constructor

- Constructors can have any number of parameters, including none
- A **default constructor** is one that takes no arguments either due to
 - No parameters or
 - All parameters have default values
- If a class has any programmer-defined constructors, it must have a programmer- defined default constructor

Default Constructor Example

```
class Square
{
    private:
        int side;

    public:
        Square()           // default
        { side = 1; }      // constructor

        // Other member
        // functions go here
};
```

Has no
parameters

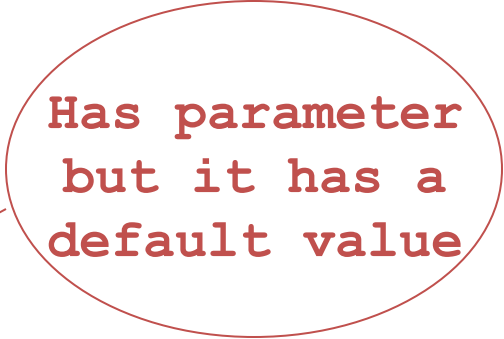


Another Default Constructor Example

```
class Square
{
    private:
        int side;

    public:
        Square(int s = 1) // default
        { side = s; }      // constructor

        // Other member
        // functions go here
};
```



Has parameter
but it has a
default value

Invoking a Constructor

- To create an object using the default constructor, use no argument list and no ()

```
Square square1;
```

- To create an object using a constructor that has parameters, include an argument list

```
Square square1(8);
```

7.7 Destructors

- Is a public member function automatically called when an object is destroyed
- The destructor name is *~className*, e.g., **~Square**
- It has no return type
- It takes no arguments
- Only 1 destructor is allowed per class
(i.e., it cannot be overloaded)

7. 8 Private Member Functions

- A **private** member function can only be called by another member function of the same class
- It is used for internal processing by the class, not for use outside of the class

7.9 Passing Objects to Functions

- A class object can be passed as an argument to a function
- When passed by value, function makes a local copy of object. Original object in calling environment is unaffected by actions in function
- When passed by reference, function can use 'set' functions to modify the object.

Notes on Passing Objects

- Using a value parameter for an object can slow down a program and waste space
- Using a reference parameter speeds up program, but allows the function to modify data in the parameter

Notes on Passing Objects

- To save space and time, while protecting parameter data that should not be changed, use a **const reference parameter**

```
void showData(const Square &s)
// header
```

- In order to for the showData function to call **Square** member functions, those functions must use **const** in their prototype and header:

```
int Square::getSide() const;
```

Returning an Object from a Function

- A function can return an object

```
Square initSquare();    // prototype  
s1 = initSquare();      // call
```

- The function must define a object
 - for internal use
 - to use with **return** statement

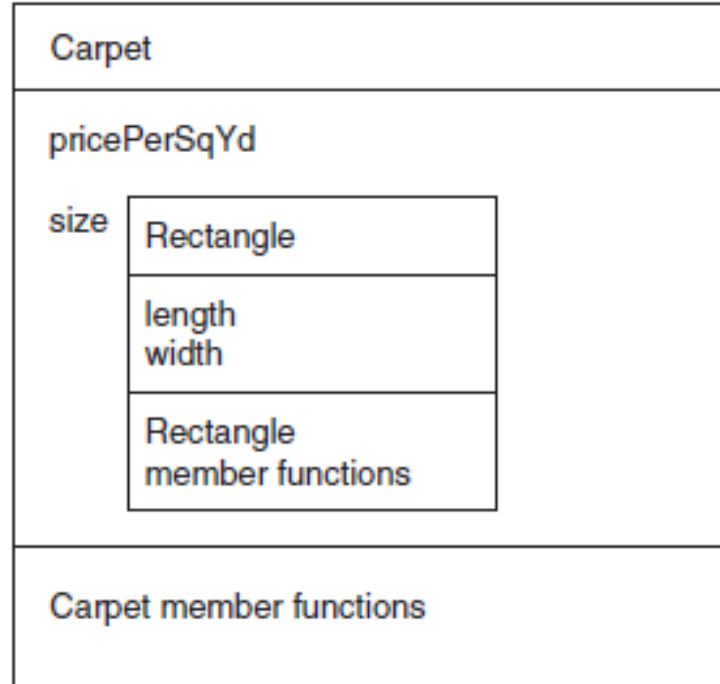
Returning an Object Example

```
Square initSquare()
{
    Square s;    // local variable
    int inputSize;
    cout << "Enter the length of side: ";
    cin >> inputSize;
    s.setSide(inputSize);
    return s;
}
```

7.10 Object Composition

- Occurs when an object is a member variable of another object.
- It is often used to design complex objects whose members are simpler objects
- ex. (from book): Define a rectangle class. Then, define a carpet class and use a rectangle object as a member of a carpet object.

Object Composition, cont.



Using Separate Files

- Place class declaration in a header file that serves as the **class specification file**. Name the file ***classname.h*** (for example, ***Square.h***)
- Place member function definitions in a **class implementation file**. Name the file ***classname.cpp*** (for example, ***Square.cpp***) This file should **`#include`** the class specification file.
- A client program (client code) that uses the class must **`#include`** the class specification file and be compiled and linked with the class implementation file.

Include Guards

- Used to prevent a header file from being included twice
- Format:

```
#ifndef symbol_name
#define symbol_name
. . . (normal contents of header file)
#endif
```

- ***symbol_name*** is usually the name of the header file, in all capital letters:

```
#ifndef SQUARE_H
#define SQUARE_H
. . .
#endif
```

What Should Be Done Inside vs. Outside the Class

- Class should be designed to provide functions to store and retrieve data
- In general, input and output (I/O) should be done by functions that use class objects, rather than by class member functions

Thanks,..

See you next week isA,..